

Performance Innovative Task:

Building a Full-Stack To-Do List Application with FastAPI

Student: Richie Mae Omandam IT3R1

1. Development Environment Setup

I ensured that the following tools and dependencies were installed on my system:

- Python 3.11+
- Node.js and npm
- Virtual environment (venv)

I structured my project directory with two main folders:

fullstack-todo-app/

```
|—— backend/    # FastAPI application
|—— frontend/   # React application
```

2.API Endpoints

- POST /tasks/ - Create new task
- GET /tasks/ - Read all tasks
- GET /tasks/{id}/ - Read single task
- PATCH /tasks/{id}/ - Update task
- DELETE /tasks/{id}/ - Delete task
- GET /tasks/filter?completed=true - Filter tasks by completion status

3. React Front-End Application

Features Implemented

- Add new tasks
- Edit existing tasks
- Delete tasks
- View all tasks
- Filter by: All, Completed, Pending
- Toggle between light and dark mode

Integration

- Used fetch() API to call the FastAPI backend
- All CRUD and filtering operations are functional via frontend
- Styled with basic CSS and React state management

5. Optimization and Documentation

Differences Between Django REST Framework and FastAPI

Feature	Django REST Framework	FastAPI
Performance	Slower due to Django stack	Faster due to async support
Learning Curve	Steeper	Easier for beginners
Type Hinting	Optional	Enforced (Pydantic)
API Docs	Manual or DRF-YASG	Automatic Swagger & ReDoc
Flexibility	Opinionated	Very flexible

Challenges and Solutions

Challenge

- CORS errors when calling API
- Deployment issues on Render
- .
- GitHub Pages refresh 404 errors
- .

Solution

- Configured CORS in FastAPI
- Added proper build and start scripts and .env setup
- Added HashRouter in React to avoid 404s

Setup Instructions: Full-Stack To-Do List Application with FastAPI and React

1. Set Up Your Development Environment

Before you begin, ensure you have the following software installed on your local machine:

- Python (3.7 or higher)
- Node.js (14.x or higher)
- pip (Python package installer)
- Virtualenv (for creating a virtual environment)

You can install the necessary tools as follows:

Install Python and Node.js

- [Download Python](#)
- [Download Node.js](#)

Install Virtualenv (if not already installed)

pip install virtualenv

2. Project Structure

Create a project directory with two subdirectories for the backend (FastAPI) and frontend (React).

```
mkdir TODO-FASTAPI
cd TODO-FASTAPI
mkdir backend frontend
```

Now, we have the following structure:

```
TODO-FASTAPI/
|
|—— backend/
|—— frontend/
```

3. Develop the FastAPI Backend

3.1. Set Up the FastAPI Backend

Create a Virtual Environment for the Backend

Inside the backend/ directory, create a virtual environment:

```
cd backend
virtualenv venv
source venv/bin/activate
```

Install FastAPI and Dependencies

Install FastAPI, Uvicorn (for running the server), SQLAlchemy (for ORM), and Pydantic (for data validation).

pip install fastapi uvicorn sqlalchemy pydantic

4. Create API Endpoints

In the backend directory, create a main.py to define your FastAPI application and endpoints for CRUD operations:

Run the FastAPI App

Start the FastAPI server with Uvicorn:

```
uvicorn main:app --reload
```

4. Develop the React Front-End

4.1. Set Up the React Front-End

Create a React App

Inside the frontend/ directory, create a React app using Create React App:

```
cd frontend
```

```
npx create-react-app .
```

Install Axios for API Communication

Install Axios to make HTTP requests to the FastAPI backend:

```
npm install axios
```

Create Components for CRUD Operations

Create components like `ToDoList`, `AddToDo`, and `ToDoItem` to handle tasks.

5. Ensure Full Integration and Functionality

- Test CRUD Operations: Ensure that the backend API works correctly with tools like Swagger UI or Postman.
- Test the Frontend: Use the React app to add, update, and delete tasks, and make sure that the state is reflected on the UI.

6. Optimize and Document Your Work

- Write Clean Code: Ensure your code is well-organized and follows best practices.
- Documentation: Write a report comparing Django REST Framework (DRF) and FastAPI, highlighting the advantages and challenges you faced during development.

7. Deploy the Application

1. Deploy Backend to Render:

- Create a new service on [Render](#) and push your FastAPI code to a GitHub repository.
- Configure the Render deployment and deploy the FastAPI backend.

2. Deploy Frontend to GitHub Pages, Netlify, or Vercel:

- Push your React frontend to a GitHub repository.
- Follow the deployment instructions for [GitHub Pages](#), [Netlify](#), or [Vercel](#).